INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	**INTRODUCCIÓN AL DESARROLLO WEB**				
TITULO DE LA PRÁCTICA:	*Express y Construcción de una API RESTful*				
NÚMERO DE PRÁCTICA:	*06*	**AÑO LECTIVO:**	*2026*	**NRO. SEMESTRE:**	*II*
FECHA DE PRESENTACIÓN	*26/10/2026*	**HORA DE PRESENTACIÓN**	*hh/mm/ss*		
INTEGRANTE (s) - *Christopher William Vásquez Vasquez* - *Torres Escalante Omar Ignacio*				**NOTA (0-20)**	
DOCENTE(s): MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.					

RESULTADOS Y PRUEBAS

I. EJERCICIO RESUELTO:

Estructura modular en Express por capas demostrada por el docente:

// controllers/productController.js

```
let products = [{ id: 1, nombre: 'Laptop', precio: 3500 }];
exports.getAllProducts = (req, res) => {res.status(200).json({ status: 'success', data: products });};
exports.createProduct = (req, res) => {
 const { nombre, precio } = req.body;
 if (!nombre || !precio) {
 return res.status(400).json({ status: 'fail', message: 'Faltan campos
obligatorios' });
 }
 const newProduct = { id: products.length + 1, nombre, precio:
parseFloat(precio) };
 products.push(newProduct); res.status(201).json({ status: 'success', data: newProduct });
};
```

// routes/productRoutes.js

```
const express = require('express');
```

```
const router = express.Router();
const productController = require('../controllers/productController');
router.get('/', productController.getAllProducts);
router.post('/', productController.createProduct);
module.exports = router;
```

// app.js

```
const express = require('express');
const productRoutes = require('./routes/productRoutes');
const app = express();
app.use(express.json()); // Middleware incorporado JSON parser
// Middleware propio Logger
app.use((req, res, next) => {console.log(`[${new Date().toISOString()}] ${req.method} en ${req.url}`);next();});
app.use('/api/v1/products', productRoutes);
app.listen(3000, () => console.log('API Express en puerto 3000'
```

II. PROBLEMA PROPUESTO:

Caso Práctico: API RESTful para la Gestión de Cursos Académicos Construir un módulo backend con Express.js para administrar el recurso "Cursos", cumpliendo con la siguiente arquitectura y endpoints:

1. Arquitectura por Capas:

- Separar estrictamente el código en las carpetas: /routes (definición de endpoints), /controllers (lógica de negocio) y /data (persistencia temporal en memoria o archivo JSON).

2. Middlewares Obligatorios:

- Middleware global incorporado: express.json().
- Middleware personalizado de Auditoría (Logger): Imprimir en consola el método, la URL y el tiempo de respuesta.
- Middleware de Manejo de Errores: Capturar errores no controlados y responder un JSON formateado con código HTTP 500.

3. Endpoints de la API REST (Recurso Cursos):

- GET /api/cursos: Retorna lista completa de cursos con código 200. Permite filtrar mediante query string (ej. /api/cursos?creditos=4).
- GET /api/cursos/:id: Busca un curso por id. Retorna 200 si existe o 404 si no se encuentra.
- POST /api/cursos: Valida los campos obligatorios (nombre, codigo, creditos). Crea el curso y retorna 201 Created.
- PUT /api/cursos/:id: Actualiza las propiedades del curso. Retorna 200 OK o 404 Not Found.
- DELETE /api/cursos/:id: Elimina el curso del arreglo local y retorna 200 OK con mensaje de confirmación o 204 No Content.

4. Pruebas y Documentación:

- Probar todos los endpoints utilizando Postman / Thunder Client y exportar la colección de pruebas en formato JSON.

REPOSITORIO
https://github.com/Christopher-Vasquez/IDWeb/tree/main/Lab06

DRIVE(Donde se encuentra un video explicando cada prueba de endpoint en POSTMAN):
https://drive.google.com/drive/folders/1wfQZLGL-12Pnr6ByQ-rTO1lm6h-s3LPA?usp=sharing

ESTRUCTURA(algunos archivos fueron omitidos para optimizar el espacio):

```
Lab06/
├── controllers/
│   └── cursoController.js
├── data/
│   └── cursos.js
├── middlewares/
│   ├── errorHandler.js
│   └── logger.js
├── node_modules/
├── postman/
│   └── Lab06_Cursos.postman_collection.json
├── public/
│   ├── index.html
│   └── scripts.js
├── routes/
│   └── cursoRoutes.js
├── .gitignore
├── app.js
├── package-lock.json
└── package.json
```

Lab06/

controllers/cursoController.js

```
const cursos = require('../data/cursos');

const validarCurso = ({ nombre, codigo, creditos } = {}) => {
    if (!nombre || !codigo || creditos === undefined || creditos === null || creditos === '')
return 'Faltan campos obligatorios: nombre, codigo y creditos';
    if (typeof nombre !== 'string' || typeof codigo !== 'string') return 'nombre y codigo
deben ser texto';
    const numCreditos = Number(creditos);
    if (!Number.isInteger(numCreditos) || numCreditos <= 0) return 'creditos debe ser un
número entero mayor que 0';
    return null;
};
```

```
exports.getAllCursos = (req, res) => {
    const { creditos } = req.query;
    let resultado = cursos;
    if (creditos !== undefined) {
        const numCreditos = Number(creditos);
        if (Number.isNaN(numCreditos)) return res.status(400).json({ status: 'fail', message:
'El filtro creditos debe ser numérico' });
        resultado = cursos.filter(c => c.creditos === numCreditos);
    }
    res.status(200).json({ status: 'success', results: resultado.length, data: resultado });
};

exports.getCursoById = (req, res) => {
    const id = Number(req.params.id);
    const curso = cursos.find(c => c.id === id);
    if (!curso) return res.status(404).json({ status: 'fail', message: `No existe un curso con
id ${req.params.id}` });
    res.status(200).json({ status: 'success', data: curso });
};

exports.createCurso = (req, res) => {
    const error = validarCurso(req.body);
    if (error) return res.status(400).json({ status: 'fail', message: error });
    const { nombre, codigo, creditos } = req.body;
    const nuevoId = cursos.reduce((max, c) => Math.max(max, c.id), 0) + 1;
    const nuevoCurso = { id: nuevoId, nombre, codigo, creditos: Number(creditos) };
    cursos.push(nuevoCurso);
    res.status(201).json({ status: 'success', data: nuevoCurso });
};

exports.updateCurso = (req, res) => {
    const id = Number(req.params.id);
    const indice = cursos.findIndex(c => c.id === id);
    if (indice === -1) return res.status(404).json({ status: 'fail', message: `No existe un
curso con id ${req.params.id}` });
    const error = validarCurso(req.body);
    if (error) return res.status(400).json({ status: 'fail', message: error });
    const { nombre, codigo, creditos } = req.body;
    cursos[indice] = { id, nombre, codigo, creditos: Number(creditos) };
    res.status(200).json({ status: 'success', data: cursos[indice] });
};

exports.deleteCurso = (req, res) => {
    const id = Number(req.params.id);
    const indice = cursos.findIndex(c => c.id === id);
    if (indice === -1) return res.status(404).json({ status: 'fail', message: `No existe un
curso con id ${req.params.id}` });
    const [eliminado] = cursos.splice(indice, 1);
    res.status(200).json({ status: 'success', message: 'Curso eliminado correctamente', data:
eliminado });
};
```

data/cursos.js

```
module.exports = [
    { id: 1, nombre: 'Introducción al Desarrollo Web', codigo: 'IDW-101', creditos: 4 },
    { id: 2, nombre: 'Estructuras Discreta I', codigo: 'ED-201', creditos: 3 },
    { id: 3, nombre: 'Cálculo I', codigo: 'CAL-301', creditos: 5 }
];
```

middlewares/

errorHandler.js

```
const errorHandler = (err, req, res, next) => {
    // JSON mal formado en el body: es un error del cliente (400), no del servidor
    if (err.type === 'entity.parse.failed') {return res.status(400).json({
       status: 'fail',
       message: 'El cuerpo de la petición no es un JSON válido'});
    }
    console.error(`[ERROR] ${req.method} ${req.originalUrl}:`, err.message);
    res.status(500).json({ status: 'error', message: 'Error interno del servidor' });
};

module.exports = errorHandler;
```

logger.js

```
const logger = (req, res, next) => {
    const inicio = process.hrtime.bigint();

    // El evento 'finish' se dispara cuando la respuesta ya fue enviada al cliente
    res.on('finish', () => {
        const ms = Number(process.hrtime.bigint() - inicio) / 1e6;
        console.log(`[${new Date().toISOString()}] ${req.method} ${req.originalUrl} ->
${res.statusCode} (${ms.toFixed(2)} ms)`);
    });

    next();
};

module.exports = logger;
```

cursos/cursosRoutes.js

```
const express = require('express');
const router = express.Router();
const cursoController = require('../controllers/cursoController');

router.get('/', cursoController.getAllCursos);
router.get('/:id', cursoController.getCursoById);
router.post('/', cursoController.createCurso);
router.put('/:id', cursoController.updateCurso);
router.delete('/:id', cursoController.deleteCurso);

module.exports = router;
```

app.js

```
const express = require('express');
const path = require('path');

const cursoRoutes = require('./routes/cursoRoutes');
const logger = require('./middlewares/logger');
const errorHandler = require('./middlewares/errorHandler');

const app = express();
const PORT = 3000;

// 1. Middlewares globales
app.use(logger);
// CORS básico: permite usar el frontend desde Live Server u otro puerto
app.use((req, res, next) => {
    res.header('Access-Control-Allow-Origin', '*');
    res.header('Access-Control-Allow-Methods', 'GET,POST,PUT,DELETE,OPTIONS');
    res.header('Access-Control-Allow-Headers', 'Content-Type');
    if (req.method === 'OPTIONS') return res.sendStatus(204);
    next();
});

app.use(express.json());

// 2. Servir archivos de la carpeta public
app.use(express.static(path.join(__dirname, 'public')));

// 3. Rutas de la API
app.use('/api/cursos', cursoRoutes);

// 4. Ruta para probar errores
app.get('/api/error', (req, res, next) => {
    next(new Error('Error de prueba provocado a propósito'));
});

// 5. Rutas inexistentes
app.use((req, res) => {
    res.status(404).json({
        status: 'fail',
        message: `La ruta ${req.originalUrl} no existe`
```

```
    });
});

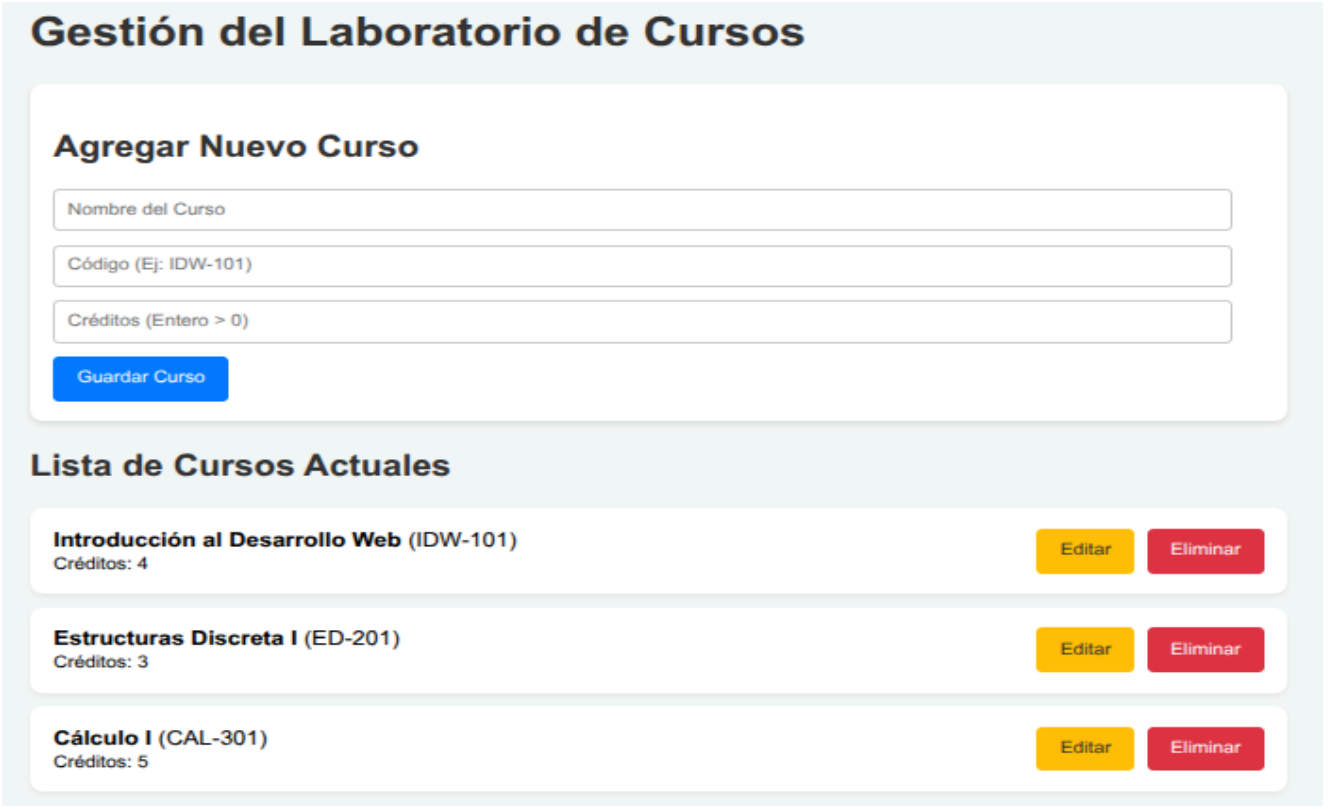
// 6. Manejador global de errores (al final)
app.use(errorHandler);

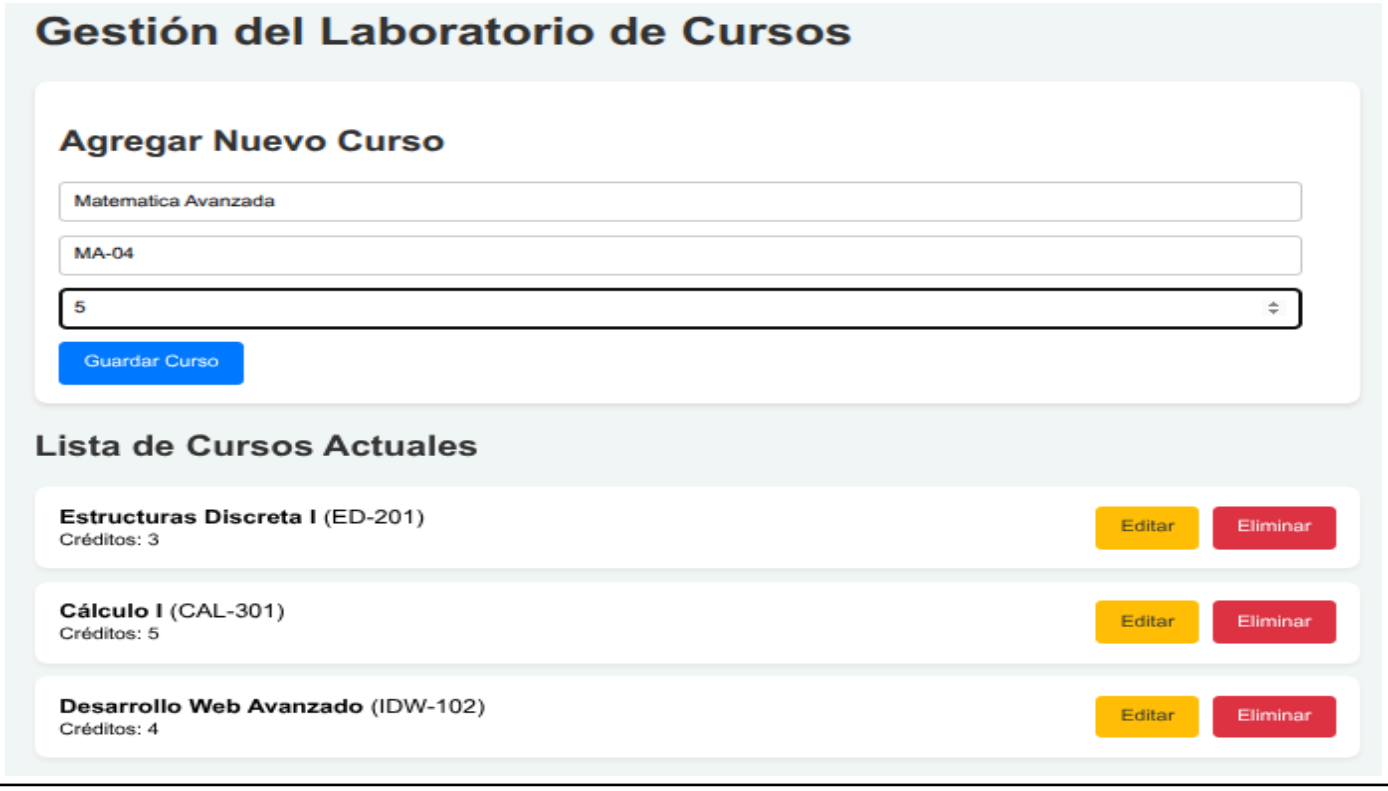
// 7. Iniciar servidor
app.listen(PORT, () => {
    console.log(`Servidor en http://localhost:${PORT}`);
});
```

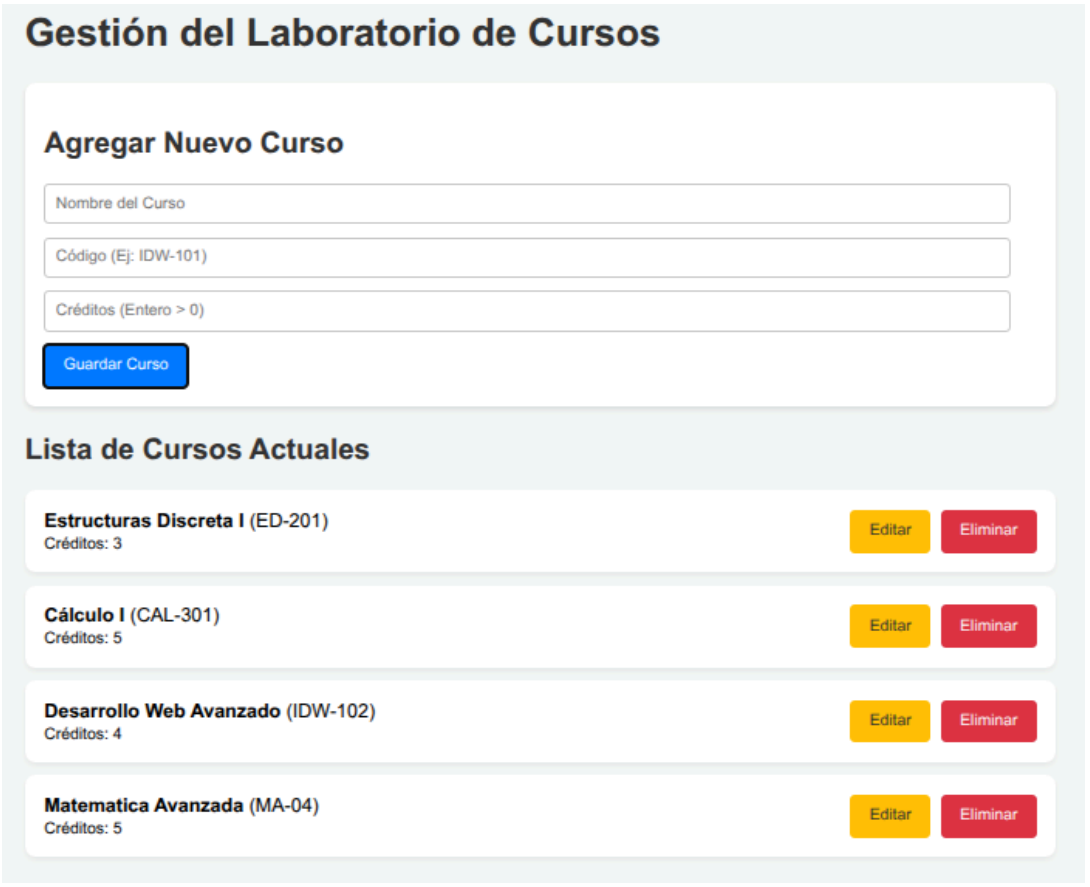
package.json

```
{
  "name": "lab06",
  "version": "1.0.0",
  "description": "Lab 06 - API RESTful de Cursos con Express (rutas, controladores y
middlewares)",
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "dev": "node --watch app.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^5.3.0"
  }
}
```

CAPTURAS DE LA PÁGINA ENCENDIDA








```
[2026-10-27T00:23:32.114Z] GET /api/cursos -> 200 (3.17 ms)
[2026-10-27T00:24:25.372Z] POST /api/cursos -> 201 (0.87 ms)
[2026-10-27T00:24:25.376Z] GET /api/cursos -> 200 (0.43 ms)
```

III. CUESTIONARIO:

IV. ¿Qué es un Middleware en Express y cuál es el impacto de no invocar la función next() dentro de su ejecución?

Middleware: Es una función que tiene acceso al objeto de solicitud (`req`), al objeto de respuesta (`res`) y a la siguiente función middleware en el ciclo de vida de la petición de la aplicación (`next`). Los middlewares pueden ejecutar código, realizar modificaciones en los objetos `req`/`res`, finalizar el ciclo de la petición o pasar el control al siguiente middleware mediante `next()`.

Si un middleware no llama a `next()` y tampoco finaliza la petición enviando una respuesta al cliente (por ejemplo, con `res.json()` o `res.send()`), la solicitud se quedará colgada (bloqueada). El servidor mantendrá la conexión abierta de forma indefinida hasta que ocurra un tiempo de espera (*timeout*), lo que provocará que el cliente nunca reciba una respuesta y la aplicación parezca congelada en esa ruta.

V. Explique la diferencia de propósito semántico y de implementación entre req.params, req.query y req.body en Express.

Propiedad	Propósito Semántico	Implementación
req.params	Identificar y recuperar un recurso específico mediante variables embebidas en la ruta.	Definidos en la ruta con dos puntos (/products/:id). Se envían directamente en la URL.
req.query	Filtrar, ordenar, buscar o paginar colecciones de recursos sin alterar la ruta principal.	Parámetros de consulta opcionales al final de la URL, después del signo de interrogación (?).
req.body	Enviar datos complejos o pesados destinados a la creación o actualización completa/parcial de un recurso.	Se envía oculto en el cuerpo (*payload*) de la petición HTTP (usualmente métodos POST, PUT, PATCH). Requiere express.json().

VI. ¿Por qué es fundamental devolver códigos de estado HTTP semánticos (200, 201, 400, 404, 500) en una API RESTful en lugar de responder siempre con código 200 OK?

- **Estandarización**: El protocolo HTTP provee un lenguaje universal mediante códigos de estado para informar al cliente sobre el resultado de su petición sin necesidad de analizar el cuerpo de la respuesta.
- **Consecuencias de usar siempre 200 OK**: Si una API responde siempre con `200 OK`, el cliente no podrá programar lógica condicional basada en el éxito o fallo de la petición de manera estándar.
- **Importancia de cada código:**
  - **`200 OK`: La solicitud estándar fue exitosa (ej. un `GET`).**
  - **`201 Created`: El recurso se creó exitosamente (ideal para `POST`, como el que hiciste en tu laboratorio).**
  - **`400 Bad Request`: El cliente envió datos mal formados o inválidos (error de validación).**
  - **`404 Not Found`: El recurso solicitado no existe en el servidor.**
  - **`500 Internal Server Error`: Ocurrió un fallo imprevisto en el servidor (ej. error de base de datos o excepción no controlada).**

CONCLUSIONES

Se implementó exitosamente una API RESTful para la gestión del recurso "Cursos" utilizando Express.js sobre Node.js, aplicando una arquitectura modular por capas que separa con claridad las rutas (`/routes`), la lógica de negocio (`/controllers`) y la gestión de datos (`/data`). Asimismo, se aprovechó el ciclo de vida de Express mediante middlewares integrados y personalizados para el parseo de JSON, el registro de auditoría (logger) y el control global de excepciones con respuestas HTTP 500, garantizando una estructura escalable y mantenible.

Mediante el uso de verbos HTTP estandarizados (`GET`, `POST`, `PUT`, `DELETE`), el manejo semántico de parámetros (`req.params`, `req.query`, `req.body`) y el retorno de códigos de estado adecuados (`200`, `201`, `400`, `404`), se logró un diseño robusto y alineado a los estándares REST. La funcionalidad completa del módulo fue validada satisfactoriamente mediante pruebas de consumo en Postman / Thunder Client, confirmando la correcta ejecución de las operaciones CRUD y el filtrado de información.

METODOLOGÍA DE TRABAJO

Para el desarrollo de la práctica se adoptó una metodología estructurada que inició con el análisis de requerimientos y el diseño de la arquitectura modular por capas en Express.js. A continuación, se implementó el backend con sus respectivos controladores, rutas y middlewares de auditoría y errores, complementado con una interfaz web básica para pruebas locales. Finalmente, se validaron los endpoints con Postman, asegurando el control de versiones mediante el repositorio en GitHub y documentando los resultados.

REFERENCIAS Y BIBLIOGRAFÍA

- *Express.js Community. (2025). Express Routing and Middleware Documentation. Expressjs.com.*
- *Restlet Team. (2023). RESTful API Design Best Practices. O'Reilly.*
- *Wilson, M. (2021). Building RESTful Web APIs with Node.js and Express. Packt Publishing.*